

```

1.2) void cadastrarCategoria (struct Categoria VC[], int *qtd) { 1,5
    if (TAM == *qtd) {
        printf("Limite de categorias cadastradas atingido.");
        return;
    }
    int ehPossivel = 1;
    while (ehPossivel) {
        printf("Digite o número do código da categoria: ");
        int codig;
        scanf("%d", &codig);
        if (buscaCategoria(VC, *qtd, codig) != -1) {
            printf("Código já cadastrado");
            continue;
        }
        printf("Digite o nome da categoria: ");
        scanf("%s", VC[*qtd].nome);
        VC[*qtd].codig = codig;
        *qtd += 1;
        printf("Produto cadastrado com sucesso.");
        ehPossivel = 0;
    }
}

```

```

2.1) void imprimirCategorias (struct Categoria VC[], int qtd) { 1,0
    for (int i = 0; i < qtd; i++) {
        printf("Código: %d", VC[i].codig);
        printf("\nNome: %s", VC[i].nome);
    }
}

```

```

int buscaCategoria (struct Categoria VC[], int qtd, int codig) {
    for (int i = 0; i < qtd; i++) {
        if (VC[i].codig == codig) {
            return i;
        }
    }
    return -1;
}

```



```

3º) void ImprimirProdutos (struct Produto vp[], int qtdProdutos, struct Categoria vc[],
int qtdCategorias) {
    for (int i = 0; i < qtdProdutos; i++) {
        printf("Codigo: %.d", vp[i].Codigo);
        printf("Titulo: %.s", vp[i].Titulo);
        printf("Descricao: %.s", vp[i].Descricao);
        printf("Categoria: %.s", vc[buscaCategoria(vp[i].Categoria, vc, qtdCategorias)].nome);
        printf("Preço: %.2lf", vp[i].preco / 100.0);
    }
}

```

✓ 2.0

```

4º) void SelectionSortPorDescricao (struct Produto v[], int qtd) {
    int memor;
    struct Produto aux;
    for (int i = 0; i < qtd - 1; i++) {
        memor = i;
        for (int j = i + 1; j <= qtd - 1; j++) {
            if (strcmp(v[j].Descricao, v[memor].Descricao) < 0) {
                memor = j;
            }
        }
        aux = v[i];
        v[i] = v[memor];
        v[memor] = aux;
    }
}

```

✓ 2.0


```

5º) int buscaBinariaPorDescricao (struct Produto V[], int qtd, char *x) {
    int inicio = 0, meio, fim = qtd - 1;
    do {
        meio = (inicio + fim) / 2;
        if (strcmp(V[meio].descricao, x) > 0) {
            fim = meio - 1;
        } else {
            inicio = meio + 1;
        }
    } while (strcmp(V[meio].descricao, x) != 0 && inicio <= fim);
    if (strcmp(V[meio].descricao, x) == 0) {
        return meio;
    }
    else if (qtd == 0) {
        return -2;
    }
    else {
        return -1;
    }
}

```

1,5